

# **AI for Economic Research: Dynamic Models, Language, and Agents**

## **Lecture 10.6b: Case Study 5 — Inside the Sprint (Execution)**

Zhigang Feng

Spring 2026

## Topic 10.6b Agenda

1. **Inside the Sprint — Execution** — Stages 2, 3, 4 and meta-commentary

## **Case Study 5 — Stages 2, 3, 4**

From a single glue index to a distribution — and an LLM-built index in between

## Stage 2 — AI Task Dispersion (opens)

**Question.** Given the mean AI exposure  $\bar{E}_o$ , does within-occupation dispersion of exposure predict labor-share movement?

**Data (new).** Add the **Eloundou et al. (2023)** GPT task-exposure list as a second, independent AI-exposure source parallel to FRS. Tasks, not abilities. 13,225 Core tasks across 872 O\*NET-SOC codes. We collapse their E0/E1/E2 exposure rubric into *our own* 0–4  $T$ -score per task (*our label, not an Eloundou-published variable*).

**Strategy.** Per-occupation features (task\_count, ai\_exposure\_sd) built at O\*NET-SOC, bridged through SOC 2018  $\rightarrow$  OCC1950 via CPS-weighted pairs. Outcome:  $\Delta$ share in an OCC1950  $\times$  (age  $\times$  education) cell.

**Report.** Coefficient tables for Experiments A–E across four cells and two sources. Binscatter figures.

**Analyze.** Nested WLS ladder weighted by pre-period employment share, HC1 SEs:

- M0:  $\Delta s \sim \bar{E}$
- M1:  $\Delta s \sim \bar{E} + SD$
- M2:  $\Delta s \sim \bar{E} + TC$
- M3:  $\Delta s \sim \bar{E} + SD + TC$

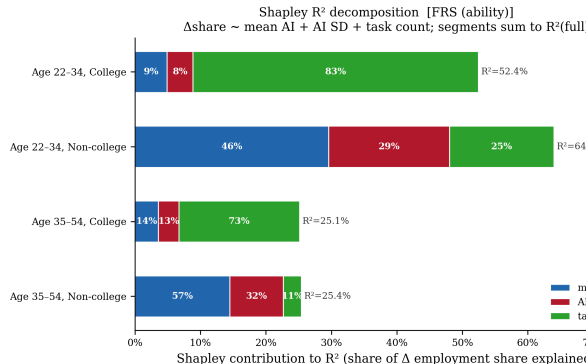
**Interpret.** Does  $|\hat{\beta}_{SD}|$  or  $|\hat{\beta}_{TC}|$  rival  $|\hat{\beta}_{\bar{E}}|$  on a 1-SD scale?

## Stage 2 — Features, Specs, and the Head-to-Head Question

Feature pairs per source

| Source           | Features                                                                |
|------------------|-------------------------------------------------------------------------|
| FRS (abilities)  | task_count, ai_exposure_sd<br>— 52 abilities screened at IM<br>$\geq 3$ |
| Eloundou (tasks) | task_count_el,<br>ai_exposure_sd_el — Core<br>tasks, T-score 0–4        |

**The head-to-head.** All features z-scored within cell so the ratio  $|\hat{\beta}_{\text{feat}} / \hat{\beta}_{\bar{E}}|$  says “how many 1-SD of  $\bar{E}$  does a 1-SD of the feature deliver?”



**Experiment E Shapley decomposition (FRS side).** Each cell's full-model  $R^2$  split across  $\{\bar{E}, \text{SD}, \text{TC}\}$  by average marginal contribution across orderings.

## Stage 2 — Three Findings

1. **Task count is robust.** Young  $\times$  College:  $\hat{\beta}_{TC} = +0.0059$  ( $p = 0.005$ ) on FRS,  $+0.0045$  ( $p = 0.002$ ) on Eloundou. Same sign, both sources, same cell. Carries 62–83% of  $R^2$  in the Shapley decomposition for that cell.
2. **AI SD is cell-specific.** Significant under Eloundou in Young  $\times$  College ( $p = 0.018$  in M3) and under FRS in Middle  $\times$  Non-college ( $p = 0.005$ ). Elsewhere indistinguishable from zero. Not universal.
3. **Mean AI exposure is not a sufficient statistic.** In 5 of 8 cells the Shapley share of the mean-AI channel is below half. The residual share is concentrated in TC or SD depending on the cell.

**Middle  $\times$  Non-college (FRS; 36.7% of prime-age employment).**  $\hat{\beta}_{\bar{E}} = -0.0040$  (M1),  $\hat{\beta}_{SD} = -0.0041$  (M1). Ratio  $|\hat{\beta}_{SD}/\hat{\beta}_{\bar{E}}| \approx 1.01$ , and SD adds +15.8pp to  $R^2$  on top of mean AI alone. In this cell, within-job dispersion matches the mean-AI channel dollar-for-dollar.

# End of Stage 2 — Memo, Critique, What Launched Stage 3

## The memo

ai\_task\_dispersion\_memo.tex  
— 14 pages, five experiments,  
descriptive throughout.

Bottom line: the task-count finding survives replication; within-job SD is a second channel with cell-specific bite; mean AI exposure is not a sufficient statistic.

## The coauthor critique

- “Task count tells you the *number* of tasks. But not whether each task is *AI-substitute* or *AI-assist*.”
- “Eloundou gives one *T*-score per task. You need two:  $e_{\text{sub}}$  and  $e_{\text{asst}}$ .”
- “Can the LLM itself do this for 13,000 tasks?”

## What launches Stage 3

*Build a task-level LLM-labeled index that decomposes AI exposure into substitution and assistance.*

This seeds Scripts 22–29 and the unbundling\_vulnerability track.

## Stage 3 — Unbundling Vulnerability (opens)

### The missing decomposition.

Both FRS AIOE and our Eloundou-based  $T$ -score deliver *one* number per task or ability. That number bundles two qualitatively different things:

- $e_{\text{sub}}$  — AI can *replace* the worker on this task;
- $e_{\text{asst}}$  — AI can *help* the worker do this task faster.

A scholar with one task at  $e = 0.8$  and a call-center agent with one task at  $e = 0.8$  have the same AIOE score — and likely very different labor-market trajectories.

### What the LLM can do that the literature can't.

The O\*NET task description plus a rubric is enough for a modern LLM to distinguish the two readings task-by-task. 13,225 tasks  $\times$  (sub, asst) scores is  $\sim 26,000$  new data points that do not exist in any public release.

### Deliverables.

- $R_o = E_{\text{sub}} / (E_{\text{sub}} + E_{\text{asst}} + \epsilon)$  — occupation-level substitution share.
- $U_o$  — composite Unbundling Vulnerability Index that uses  $R_o$ .

The LLM can score something the original index was never built to see. Home: docs/unbundling\_vulnerability/.

## Stage 3 — The Ambition: Scoring 13,225 Tasks

|                        |                                                                   |
|------------------------|-------------------------------------------------------------------|
| Corpus size            | 13,225 O*NET Core tasks                                           |
| Per-task output        | $(e_{\text{sub}}, e_{\text{asst}}, \text{rationale})$ in $[0, 1]$ |
| Calls required (naïve) | 13,225 LLM calls                                                  |
| Budget at Claude Max   | one 5-hour window                                                 |
| Strategy               | <b>bulk + adjudicate + cache</b>                                  |

**Three design moves** that turn a 13k-call ambition into a tractable pipeline:

- **Bulk:** 10 tasks per Haiku call — one JSON array in, one JSON array out. Cuts calls by  $10\times$ .
- **Adjudicate:** Sonnet reruns the tasks Haiku flagged as borderline.
- **Cache:** one file per task on disk; re-runs skip already-labeled tasks. Interruption-proof.

### Why each move matters

Without bulking:  $13\text{k calls} \times \sim 10\text{s} \approx 36\text{ hours}$  — no single Claude Max window fits.

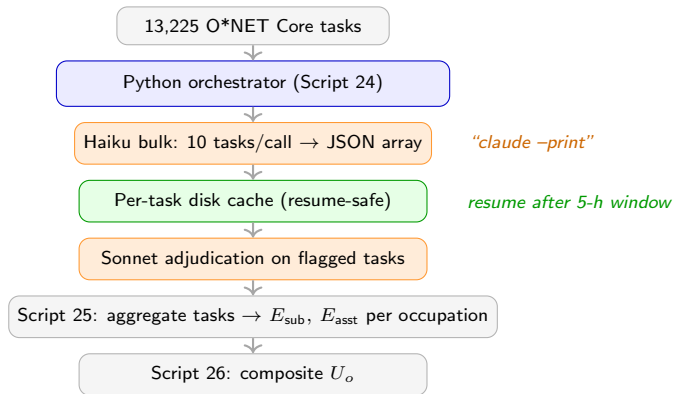
Without adjudication: Haiku's borderline judgments carry the whole index; no second opinion, no way to flag hard cases.

Without caching: any network blip, quota limit, or power loss destroys hours of work. The 5-hour window *will* expire mid-run — the cache turns that from a catastrophe into a coffee break.

### Script home.

`scripts/24_label_substitution_assistance.py`

## Stage 3 — The Labeling Architecture



**Three layers of LLM.** Claude Code writes Python (layer 1); the Python spawns `claude -print` Haiku sessions (layer 2); Sonnet adjudicates the borderline cases (layer 3). **11,171 of 13,225 tasks labeled (84.5%)** in one 5-hour window before the quota expired. Resume via cache; no task re-labeled.

## Stage 3 — The Rubric Behind the Labels

### Replacement score $e_{\text{sub}}$

- 0.00: AI cannot replace the task
- 0.25: AI handles only narrow pieces
- 0.50: human and AI split the task
- 0.75: AI does most of the work; human reviews
- 1.00: AI replaces the task almost entirely

### Assistive score $e_{\text{asst}}$

- 0.00: no productivity gain
- 0.25: speeds up a narrow sub-step
- 0.50: roughly doubles throughput
- 0.75: handles most of the time cost
- 1.00: task becomes trivial once AI is used

**Three anchors.** Clear substitute: *transcribe audio*. Clear assistant: *perform laparoscopic surgery*. Borderline case: *draft a legal contract clause*. Input was a batch of 10 tasks; output was structured JSON with scores, rationale, and an ambiguity flag.

## Stage 3 — Token Engineering at Scale

### Operational trick

Run `claude -print` from a clean temporary directory so the project's 12k-token `CLAUDE.md` does *not* auto-load into every bulk call.

**Why it matters.** 13k tasks  $\Rightarrow$  thousands of repeated calls; small prompt waste becomes massive token waste; cheap calls stay cheap only if context stays minimal.

### Economic interpretation

**Saved scale.** About 160M tokens avoided over the full run. At this scale, prompt engineering is not cosmetic; it is cost control and latency control.

**Teaching point.** The rubric stayed constant; the execution environment changed. Agentic AI is strongest when the human specifies the rule once and the system executes it thousands of times.

## Stage 3 — Running Massive Work Unsupervised

### What the economist did.

- Wrote the rubric (~30 lines) and three calibration anchors.
- Read ~12 LLM-returned labels by eye to sanity-check the scale.
- Launched the run and walked away.

### What Claude Code did.

- Wrote the orchestrator (Script 24).
- Spawned thousands of `claude -print Haiku` sessions.
- Wrote each task's result to its own cache file.
- Detected quota expiration, logged cleanly, exited gracefully.

**What the LLM did.** Returned ( $e_{\text{sub}}$ ,  $e_{\text{asst}}$ ) for 11,171 tasks; flagged 3,456 for adjudication.

### Log excerpt (`script24_bulk_run.log`):

```
06:41:22 batch 1108 -> 10 cached
...
11:28:03 rate-limit backoff (45s)
11:29:57 ERROR claude-max window expired
11:29:57 cache: 11,171/13,225 tasks
```

**Supervision cost.** ~ 20 min of economist time to spec the rubric; ~ 5 h of unsupervised LLM work; ~ 12 labels read by eye out of 11,171. This is the ratio that makes LLM labeling a viable empirical tool at corpus scale.

## Stage 3 — From per-Task Scores to $U_o$

**Task** → **occupation aggregation** (Script 25):

$$E_o^{\text{sub}} = \sum_t w_t \cdot e_t^{\text{sub}}, \quad E_o^{\text{asst}} = \sum_t w_t \cdot e_t^{\text{asst}}$$

$$R_o = \frac{E_o^{\text{sub}}}{E_o^{\text{sub}} + E_o^{\text{asst}} + \epsilon}$$

**Composite** (Script 26):

$$U_o = \alpha \bar{E}_o + \beta V_o + \theta T_o + \lambda (R_o \cdot \bar{E}_o) - \gamma C_o - \mu Q_o$$

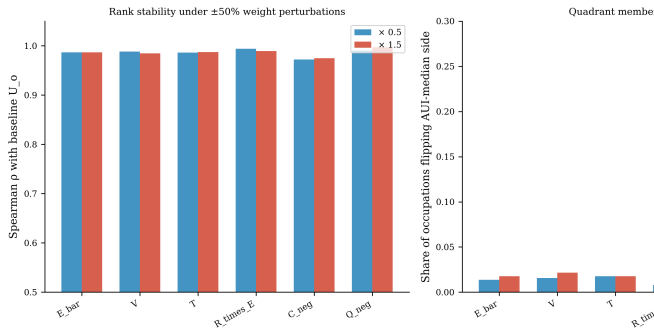
**Interpretation.**  $V$  is within-job dispersion,  $T$  is top-tail exposure,  $C$  is coordination cost, and  $Q$  is accountability intensity. The point is not that one scalar is perfect; it is that the index records several reasons a job may be vulnerable or protected.

## Stage 3 — Weight Sensitivity of $U_o$

### Declared weights

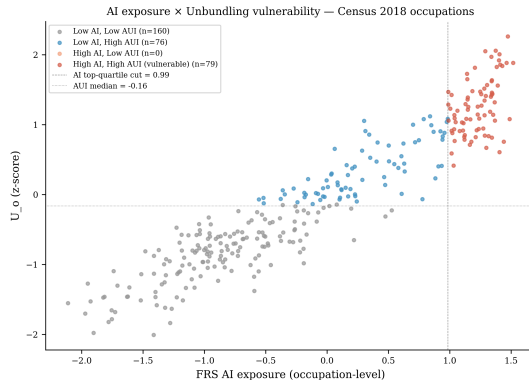
(config.py:AUI\_WEIGHTS, not data-fit):

```
AUI_WEIGHTS = {  
    "E_bar": 0.30,  
    "V": 0.15,  
    "T": 0.20,  
    "R_times_E": 0.15,  
    "C_neg": 0.15,  
    "Q_neg": 0.05,  
}
```



**Stability check.** The top and bottom occupations in  $U_o$  survive reasonable re-weighting. That does not prove the weights are correct; it shows the ranking is not driven by one arbitrary coefficient choice.

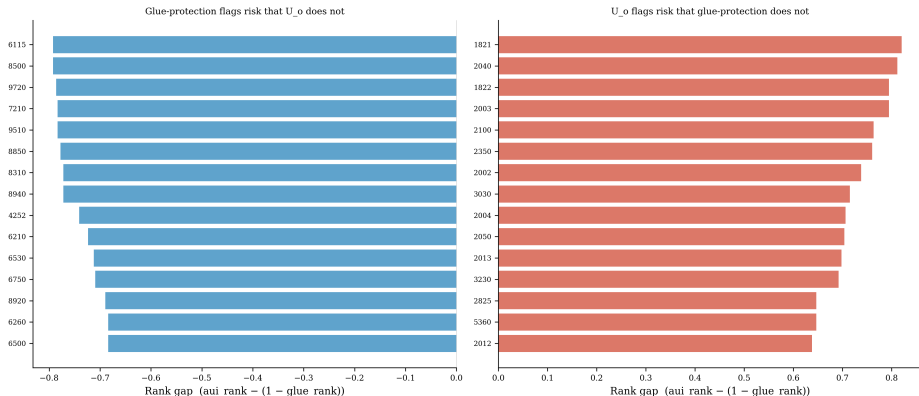
## Stage 3 — Reading the $U_o$ Index



**Quadrants.** Top-right occupations are wholesale-substitution candidates. Top-left occupations face reskilling pressure even when mean exposure is not especially high. The decomposition changes the stories at the margin, not only at the extremes.

## Stage 3 — Where $U_o$ Changes the Ranking

Where  $P^{\text{rich}}$  and  $U_o$  disagree



**Disagreement region.** The interesting cases are where  $U_o$  and  $P^{\text{rich}}$  disagree. That is exactly where the task-level labeling work pays off: it adds information the Stage 1 protection index could not recover.

# End of Stage 3 — Memo, Critique, and Cost

## The memo (partial)

unbundling\_vulnerability\_empirical\_s  
is populated *partially* because  
Stage 3 paused with 2,054  
remainder + 3,456 adjudication  
queued.

The design memo + LLM prompts  
+ quadrant figures are complete;  
 $R_o$  numbers are placeholder pend-  
ing the remaining labels.

## The coauthor critique

Apr 22 call, Xiaojie Liu:

*"Your top-tail share is 0.97 corre-  
lated with mean exposure. You  
still do not have an independent  
axis. The within-job shape of the  
distribution, not the upper tail, is  
where you can learn something  $\bar{E}$   
does not carry."*

## What launches Stage 4

*Refine within-job distribu-  
tion features; test each one  
against mean AI exposure.*

## Cost accounting.

~11k tasks  $\times$  ~2 Haiku calls  
 $\times$  ~300 tokens  $\approx$  7M tokens in  
one 5-h window. Cache guaran-  
tees no re-runs.

## Stage 4 — Exposure Structure Refinement (opens)

**Question.** Holding mean AI exposure  $\bar{E}$  fixed, which *distribution-shape* statistics of within-job exposure add information for labor-share outcomes?

**Data (no new source).** Same FRS + Eloundou inputs. What changes is the feature set: instead of just mean and SD, compute a family of shape statistics.

### Strategy.

- Tighten tail thresholds:  $p_{33} \rightarrow p_{10}$  (FRS),  $T \geq 3 \rightarrow T = 4$  (Eloundou).
- Add `top_bottom_gap`, `top_within_mean`, `bottom_quantile_mean`, `importance_rank_slope`.
- Regress  $\Delta\text{share}$  on each feature (conditioning on  $\bar{E}$ ), pooled and per-cell.

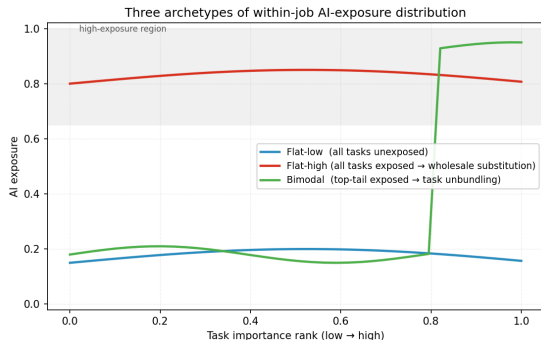
### Report.

- Correlation-with- $\bar{E}$  table for every feature.
- F1 pairwise  $\Delta R^2$  rankings.
- F5 focused horse race:  $\beta(X)/\beta(\bar{E})$  ratios.
- Shapley-on-4 allocating the increment over the  $\bar{E}$ -only baseline.

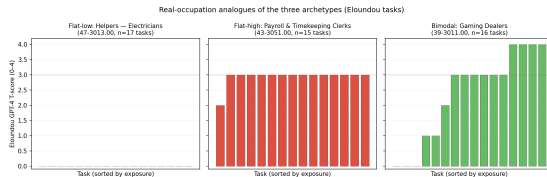
**Analyze.** Cluster-robust SEs at occ1950 for pooled specs (each occupation contributes four cell-rows; HC1 understates). Script `21b_run_exposure_structure_regressions.py`.

**Interpret.** Which refined features beat  $\bar{E}$  in magnitude? Which disappear once  $\bar{E}$  is controlled?

# Stage 4 — Three Archetypes of Within-Job Exposure



Stylized



Real analogues

**Conceptual point.** Flat-low, flat-high, and bimodal jobs can share the same mean exposure while implying three different labor-market predictions: untouched, wholesale substitution, and unbundling. **Examples:** Helpers–Electricians look flat-low; Payroll and Timekeeping Clerks look flat-high; Gaming Dealers look bimodal.

## Stage 4 — The Refined Feature Set

| Feature                | What it measures                          | $r^2$ on $\bar{E}$ (FRS)                       |
|------------------------|-------------------------------------------|------------------------------------------------|
| top_bottom_gap         | top-decile mean — bottom-decile mean      | <b>0.001</b> (nearly orthogonal to $\bar{E}$ ) |
| glue_protection_z      | $P^{\text{rich}}$ from Stage 1            | 0.040                                          |
| ai_exposure_cv         | SD / mean                                 | 0.046                                          |
| top_tail_mean_frs_p33  | mean exposure above 67th pct              | 0.049                                          |
| task_count             | number of relevant tasks                  | 0.072                                          |
| top_tail_share_frs_p10 | weighted share above 90th pct (tightened) | 0.882                                          |
| top_tail_share_frs_p33 | weighted share above 67th pct (legacy)    | 0.933                                          |

```
# config.py, added 2026-04-22
TOP_TAIL_PERCENTILE_FRS_STRICT    = 0.10    # upper-decile (strict)
TOP_TAIL_PERCENTILE_FRS_LEGACY    = 1/3      # upper-tercile (legacy)
TOP_TAIL_THRESHOLD_ELOUNDOU_STRICT = 4       # T-score == 4 (strict)
BOTTOM_QUANTILE                   = 0.10      # symmetric bottom-decile
```

## Stage 4 — F5 Horse Race vs. the Literature's $\bar{E}$

Pooled M4:  $\Delta\text{share} \sim \bar{E} + 4 \text{ features} + \text{cell FE}$ . Cluster-robust SE at occ1950. All z-scored.

| Source   | Variable                     | $\hat{\beta}$ | $p$          | $ \hat{\beta}/\hat{\beta}_{\bar{E}} $ | Note                                           |
|----------|------------------------------|---------------|--------------|---------------------------------------|------------------------------------------------|
| FRS      | $\bar{E}$ (literature index) | -0.00125      | 0.041        | 1.00                                  | baseline                                       |
| FRS      | <b>task count</b>            | +0.00268      | <b>0.001</b> | <b>2.13×</b>                          | <i>opposite sign, 2× bigger</i>                |
| FRS      | AI CV                        | -0.00183      | 0.038        | 1.46×                                 | same sign                                      |
| FRS      | top-bottom gap               | +0.00052      | 0.505        | 0.41×                                 | not sig.                                       |
| FRS      | glue protection              | -0.00011      | 0.826        | 0.09×                                 | absorbed by $\bar{E}$                          |
| Eloundou | $\bar{E}$ (literature index) | -0.00325      | 0.019        | 1.00                                  | baseline                                       |
| Eloundou | <b>AI CV</b>                 | -0.00296      | <b>0.021</b> | <b>0.91×</b>                          | <i>same sign, matches <math>\bar{E}</math></i> |
| Eloundou | task count                   | +0.00152      | 0.069        | 0.47×                                 | borderline                                     |
| Eloundou | top-bottom gap               | -0.00088      | 0.223        | 0.27×                                 | not sig.                                       |
| Eloundou | glue protection              | -0.00042      | 0.487        | 0.13×                                 | absorbed by $\bar{E}$                          |

**Reading.** With every feature on a 1-SD scale,  $|\hat{\beta}_{\text{feat}}/\hat{\beta}_{\bar{E}}|$  says “how many 1-SD of  $\bar{E}$  does a 1-SD of the feature deliver?” **Task count beats  $\bar{E}$  by 2× on FRS (opposite sign); AI CV is 1-for-1 with  $\bar{E}$  on Eloundou.** Mean AI exposure is significant, but not the largest channel.

## Stage 4 — Shapley-on-4: Where the Extra Explanatory Power Sits

Shares of the  $M_4 - M_0$  gap in  $R^2$

|          | task count | CV  | gap | glue |
|----------|------------|-----|-----|------|
| FRS      | 92%        | 5%  | 2%  | 1%   |
| Eloundou | 39%        | 38% | 21% | 2%   |

**FRS.** Task count is essentially the whole story. The other three features are absorbed into  $\bar{E}$ .

**Eloundou.** A two-channel story: task count and CV share the top almost equally, with the gap adding a real third contribution. Glue protection is near zero on both sides.

Glue protection washes out at OCC1950 pooled — but it performed well at Census 2018 in the main cross-section pipeline (Stage 1). **Which axis an index reads correctly is scale-dependent.** The honest reading: glue protection is a Census-2018 object, not a long-run OCC1950 object.

### What the regression bought.

- Joint  $F$  on the four features:  $F(4) = 5.41$ ,  $p = 0.0006$  on FRS;  $F(4) = 6.76$ ,  $p = 0.0001$  on Eloundou. The four together clearly add information beyond  $\bar{E}$ .
- The Shapley decomposition is how we know which feature is doing the work.

# End of Stage 4 — Memo and Open Threads

## The memo.

docs/exposure\_structure/exposure\_structure\_employment\_snapshot.md

— variable definitions (self-contained), feature correlations with  $\bar{E}$ ,  $V \times \bar{E}$  non-linearity, F1–F5 regression block, ratio tables.

## What survived.

- task\_count (FRS) and ai\_exposure\_cv (Eloundou) add information beyond  $\bar{E}$  at the pooled aggregation.
- top\_bottom\_gap and glue\_protection\_z wash out at OCC1950 but may still matter cell-specifically or at Census 2018.

## Open threads (explicit).

Stage 3 All-Minneapolis.md  
• Stage 3 All-Minneapolis.md 16% short. Resume from cache to finish 2,054 + 3,456 labels.

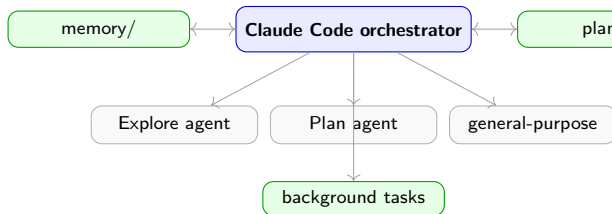
- Labor-market outcome regressions not yet re-run with the completed  $U_o$ .
- No causal design — still descriptive motivation for the theory paper.
- Composite AUI weights are declared, not fit. A data-driven re-weighting (future stage) is an option the snapshot flags but does not take.

Each row is where the next sprint starts. The list of open threads *is* the stage's deliverable, just as much as the snapshot itself.

## **Case Study 5 — Meta-Commentary**

Agentic AI under the hood, and three ways to use an LLM

# Agentic AI Under the Hood



## What ran autonomously during the 8 weeks.

- **Plan mode.** Each stage drafted, edited, user-approved, executed. Plans live at `~/ .claude/plans/*.md` across sessions.
- **Sub-agents.** Explore agent spawned in parallel (sometimes 3 at once) to map the codebase before designing new scripts.
- **Memory.** Per-project files (`project_*.md`, `MEMORY.md`) persisted what was done and where the track paused.
- **Background tasks.** Scripts 19 and 21b runs used `run_in_background + Monitor`, so planning continued while regressions compiled.

None of these were explicitly requested turn-by-turn. Claude Code *chose* when to launch a sub-agent, when to save a memory, when to start a background task. The economist's job shifted from "drive each step" to "approve each plan, critique each memo."

# Where Reusable Skills and Reviewer Agents Would Tighten the Loop

| Proposed skill/agent      | What it would do                                                                         | Where it saves time                                                        |
|---------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| index-builder (skill)     | Emit a new Script 2X with caching, z-scoring, and claims-discipline notes baked in.      | Stage 1, 2, and 4 script scaffolding.                                      |
| regression-runner (skill) | Given an outcome, features, and cells, emit the full F1–F5 regression ladder.            | Stage 2 and Stage 4 regression boilerplate.                                |
| label-corpus (skill)      | Wrap the bulk + adjudicate + cache pattern for large task-labeling jobs.                 | Turns similar corpus tasks into 1-day projects instead of 1-week projects. |
| memo-writer (agent)       | Enforce the six-element template on every end-of-stage memo and auto-link prior outputs. | Stage-close memos across the whole project.                                |
| coauthor-critic (agent)   | Read the memo and surface the top missing checks, as a simulated reviewer.               | Replaces part of the coauthor burden; never all of it.                     |

**Interpretation.** The biggest gains now come from packaging repeated research patterns, not from asking the model to be smarter in a single conversation. Claude Code was already sufficient without custom tooling — the next 10× gain would come from making these reviewer/scaffold patterns reusable.

# The Power of LLMs in this Project — Three Uses

## 1. LLM as pair programmer

Writes Python, runs it, reads the output, iterates on user prompts. The entire pipeline (Scripts 01–30, 21b) falls here. Familiar from Cases 1–4.

Where it fires: every new script, every diagnostic, every figure.

## 2. LLM as reviewer

Critiques the empirical snapshot, proposes the next design. Stage 1's self-review memo (54 KB); the Apr 22 distillation of the coauthor critique into the Stage 4 question.

Where it fires: every end-of-stage frame in this deck.

## 3. LLM as empirical data producer

Stage 3: per-task (e\_sub, e\_asst) labeling. 11,171 / 13,225 tasks labeled autonomously.

An LLM-labeled dataset of  $10^4$  observations is a first-class empirical artifact — versionable, cross-validatable, publishable.

**The third use is the one most often missed.** A student who has only seen LLMs as writing assistants has not yet seen LLMs doing labor that was previously the province of thousands of MTurk workers or hundreds of RA-hours. That is the step change this case study is built around.

# Ten Lessons from Case Study 5

*From Stage 1 (first sprint):*

1. **Lead with the puzzle, not the method.** First turn names what is unresolved; the method is downstream.
2. **Empirics first buys theoretical honesty.** The data are the cheapest critic the theory will ever meet.
3. `config.py` + **STOP GATES** = audit-grade pipeline: parameters on disk, inspections at named checkpoints.
4. **When the interaction flips sign, audit the proxy before revising the theory.** A bad measure can imitate a broken mechanism.
5. **A one-page snapshot** costs less than one rerun and compounds more than any single figure.

*From Stages 2–4 (this extension):*

6. **The first sprint is never the last.** End-of-stage memos + honest coauthor critiques are the lowest-cost lever for the next sprint.
7. **Orthogonality to  $\bar{E} \neq$  predictive power.** `top_bottom_gap` has  $r^2$  on  $\bar{E}$  of 0.001 but fires only in one cell. `task_count` is correlated with  $\bar{E}$  but wins the FRS horse race.
8. **Joint F-tests understate power under multicollinearity.** Shapley  $R^2$  decomposition separates collinear signal from noise.
9. **LLMs are economically viable corpus labelers.** Bulk (Haiku) + adjudication (Sonnet) + per-task cache = 11k tasks labeled in one 5-h window.
10. **Index design is scale-dependent.** `glue_protection_z` survives at Census 2018 but washes out at OCC1950 pooled. Declare the aggregation up-front.

**Transferable pattern.** Replace “glue work + AI labor market” with any puzzle that has (i) a theoretical primitive, (ii) a public dataset, (iii) a visible post-event window, and (iv) a task corpus an LLM can score. The same four-stage scaffold runs on any of them.